

Lab 3: Paxos — Design Document

Preface

Goals

- Provide a **linearizable** replicated key/value service. From the perspective of the clients, the system is indistinguishable from a single, correct sequential server.
- Achieve agreement on the **order of client commands** by replicating a shared log across all servers using multi-instance Paxos.
- Continue to make progress as long as a **majority of servers** can communicate with each other and with the client.
- Garbage-collect log entries that have been executed on every server, so memory usage stays bounded.
- Run correctly with any group size, including a single-server group (where every operation completes in one step).

Desired Fault Model

- Tolerate the permanent crash of any **minority** of servers (up to $\lfloor N/2 \rfloor$ out of N).
- Tolerate arbitrary message delays, drops, reorderings, and duplications.
- Tolerate network partitions: a partitioned minority remains correct (cannot make progress) and catches up automatically once the partition is resolved.
- **Not tolerated:** simultaneous crash of a majority of servers; corrupted messages; Byzantine failures; server restart with persistent state (servers do not recover).

Challenges

- **No master view server.** Unlike Lab 2, there is no central authority deciding who is in charge. Servers must elect a leader among themselves and re-elect when the leader fails.
- **Holes in the log.** A leader completing phase 1 may discover values for some slots but not others (for example, slots 1, 3, 5 have proposals but 2, 4 do not). The leader must fill the gaps with no-ops to make forward progress without skipping execution.
- **Concurrent leaders.** Network partitions or stale messages may cause two servers to simultaneously believe they are leader. Paxos's two-phase structure (with monotonically increasing ballots) guarantees safety, but the protocol must handle the resulting message storms gracefully.
- **Exactly-once semantics across replicas.** A retried client request must not be appended to the log multiple times under different slots. We use the AMOApplication wrapper plus a "have I already seen this command" check on the leader.
- **Log truncation under partition.** Garbage collection must not discard a slot that a partitioned minority has not yet executed, otherwise that minority cannot catch up when reconnected.

- **State-space explosion.** Even small Paxos clusters generate enormous state spaces under the search tester. The protocol must be carefully written to avoid unnecessary old data.

Assumptions

- The set of servers is **fixed** and known to every node at startup.
 - Every `Address` is `Comparable<Address>`, providing a deterministic total order used for ballot tie-breaking.
 - The underlying `Application` is deterministic: applying the same command sequence on every server yields identical state.
 - Servers do not recover from crashes; therefore no on-disk persistence is required.
 - Clients retry indefinitely until they receive a response.
-

Protocol

Kinds of Nodes

PaxosServer

- A single kind of node that simultaneously plays all PMMC roles (replica, acceptor, leader/proposer). State for each role is colocated to enable optimizations (for example, including both accepted and decided values in P1B replies).
- Temporary roles:
 - **Active leader:** This server believes its ballot is the highest in the system. It proposes commands, sends Heartbeats, and drives GC.
 - **Follower:** This server believes some other server is the active leader. It acts only as an acceptor, drops client requests, and watches for the leader to die through heartbeat-check mechanism.
 - **Candidate:** This server is currently running phase 1, sending P1As and collecting P1Bs in an attempt to become the active leader.

PaxosClient

- Broadcasts each command to every server and waits for a `PaxosReply`. Uses a `ClientTimer` to retry on timeout.
-

Auxiliary Types

Ballot (*defined as a static inner class*)

Field	Type
<code>seqNum</code>	integer (monotonically increasing per server)
<code>address</code>	Address (the server that proposed this ballot)

Ballots are compared first by `seqNum`, then by `address` (using `Address.compareTo`). This produces a total order across all (seqNum, server) pairs, satisfying Paxos's requirement that no two ballots ever compare equal. The class must be `static`, `Serializable`, and override `equals`/`hashCode`. The starting ballot is `Ballot(0, self)`.

LogEntry

- One per slot in the local log.

Field	Type	Meaning
<code>status</code>	enum {EMPTY, ACCEPTED, CHOSEN, CLEARED}	Lifecycle of this slot
<code>command</code>	optional AMOCommand	The command at this slot (absent when status == EMPTY or CLEARED; may be a no-op)
<code>ballot</code>	optional Ballot	The ballot under which <code>command</code> was accepted (absent when status != ACCEPTED)

When `status == CHOSEN`, the command is decided and immutable. `CLEARED` means the slot was garbage-collected and its memory is freed. The log itself is conceptually a `map<integer slot → LogEntry>`, allowing slots to be CLEARED individually.

No-op

- A distinguished `AMOCommand` value that, when executed, does nothing. The leader uses no-ops to fill log holes during phase 1 reconciliation.

State of PaxosServer

Field	Type	Initial Value	Meaning
<code>servers</code>	array of Address	provided at construction	Fixed group membership
<code>app</code>	AMOApplication	empty	Wrapped key/value application
<code>log</code>	Map<Integer, LogEntry>	empty	The replicated log (per-slot state)
<code>slotIn</code>	Integer	1	Next slot the leader will use for a new proposal
<code>slotOut</code>	Integer	1	Next slot to execute against <code>app</code> (everything below has been applied)
<code>myBallot</code>	Ballot	<code>Ballot(0, self)</code>	Highest ballot this server has ever generated
<code>promisedBallot</code>	Ballot	<code>Ballot(-1, self)</code>	Highest ballot this server has ever accepted (acceptor state)
<code>leaderState</code>	enum {ACTIVE, FOLLOWER, CANDIDATE}	FOLLOWER	This server's current leader role
<code>p1bResponses</code>	Map<Address, P1B>	empty	Collected P1Bs while candidate (cleared when election ends)
<code>p2bResponses</code>	Map<Integer, Set >	empty	Per-slot set of addresses that have accepted under <code>myBallot</code>
<code>proposedCommands</code>	Set	empty	Commands the leader has already proposed (dedup)
<code>missedHeartbeats</code>	integer	0	Counter of consecutive <code>HeartbeatCheckTimer</code> fires without a Heartbeat from the active leader. Election triggers at 2.
<code>executedSlots</code>	Map<Address, Integer>	all entries = 0	Per-server highest <code>slotOut</code> reported via heartbeat replies (leader only)
<code>garbageBoundary</code>	Integer	0	Highest slot known to be executed on every server; everything $\leq$ this is CLEARED

Constraints on evolution: `myBallot` and `acceptedBallot` are monotonically non-decreasing. `slotOut` is monotonically non-decreasing. Once a slot's status becomes `CHOSEN`, its `command` never changes. Once `CLEARED`, it stays `CLEARED`. `missedHeartbeats` is reset to 0 on every accepted Heartbeat and is meaningful only while `leaderState != ACTIVE`.

Self-message rule: When the local server is both sender and intended receiver of a Paxos message, the message is **not** sent over the network. Instead, the corresponding handler is invoked directly. This applies everywhere broadcast is used.

Messages

PaxosRequest (PaxosClient → all PaxosServers)

- **Contents:** {amoCommand} where amoCommand = {clientId, sequenceNum, operation}.
- **When sent:** Spontaneously by the client (broadcast to every server). Retried on ClientTimer.
- **At PaxosServer on receipt:**
 1. If leaderState != ACTIVE, drop silently.
 2. **Dedup:** If amoCommand is already in proposedCommands, in log (under any slot), or already executed by app, do not re-propose. If the command is already executed and a result is known, reply with PaxosReply{result} directly; otherwise drop.
 3. Otherwise, assign the command to slot slotIn, store LogEntry{ACCEPTED, amoCommand, myBallot} at that slot, increment slotIn, add to proposedCommands, and broadcast P2A{myBallot, slot, amoCommand} (skipping the network for self).

PaxosReply (PaxosServer → PaxosClient)

- **Contents:** {amoResult}.
- **When sent:** When a server executes (or has previously executed) a command from this client during log replay. Sent directly to the client whose clientId is in the command.
- **At PaxosClient on receipt:** Match sequenceNum against pending request; if it matches, deliver result and clear pending state. Otherwise ignore.

P1A (CANDIDATE → all PaxosServers)

- **Contents:** {ballot} — the candidate's myBallot.
- **When sent:** When this server transitions to CANDIDATE (heartbeat-check fired twice without seeing the active leader). Retried via LeaderElectionTimer if quorum of P1Bs not yet received.
- **At PaxosServer on receipt:**
 1. If ballot < promisedBallot, reply with P1B{promisedBallot, ∅} (a "no" vote that includes the higher ballot so the sender backs off).
 2. Otherwise, set promisedBallot = ballot, set leaderState = FOLLOWER (back off if we were also a candidate), reset missedHeartbeats = 0, and reply with P1B{ballot, accepted_log_entries} where accepted_log_entries is the local log restricted to ACCEPTED and CHOSEN slots, including each slot's status, command, and ballot.

P1B (PaxosServer → CANDIDATE)

- **Contents:** {ballot, log_entries} where ballot is the ballot the responder is acknowledging, and log_entries is a map from slot to (command, ballot, status).
- **At candidate on receipt:**
 1. If ballot != myBallot, ignore (stale).
 2. If ballot.seqNum > myBallot.seqNum (the responder included a larger ballot to indicate rejection), back off: set leaderState = FOLLOWER, advance myBallot.seqNum past the rejecting one (kept for next election), do not retry phase 1 immediately.

3. Add to `p1bResponses[sender]`.
4. If `|p1bResponses| >= [N/2] + 1`: **merge logs** (see "Log merging" below), transition `leaderState = ACTIVE`, set `promisedBallot = myBallot`, broadcast a `Heartbeat`, and broadcast `P2A` for every ACCEPTED slot resulting from the merge.

P2A (ACTIVE leader → all PaxosServers)

- **Contents:** `{ballot, slot, command}`.
- **When sent:** By the active leader for each newly proposed command, and during the post-election broadcast for every slot it is now responsible for. Retried via `AcceptTimer` until a quorum of P2Bs is received or the slot becomes CHOSEN.
- **At PaxosServer on receipt:**
 1. If `ballot < promisedBallot`, reply with `P2B{promisedBallot, slot, rejected=true}`.
 2. Set `promisedBallot = ballot`.
 3. If the slot is already `CHOSEN` locally, reply with `P2B{ballot, slot, rejected=false}` immediately (already chosen — we won't re-accept, but we acknowledge so the leader makes progress).
 4. Otherwise, store `LogEntry{ACCEPTED, command, ballot}` at this slot (overwriting any lower-ballot entry), and reply with `P2B{ballot, slot, rejected=false}`.

P2B (PaxosServer → ACTIVE leader)

- **Contents:** `{ballot, slot, rejected}`.
- **At leader on receipt:**
 1. If `ballot != myBallot` or `leaderState != ACTIVE`, ignore.
 2. If `rejected == true` and the responder included a higher ballot, back off: `leaderState = FOLLOWER`, update `myBallot.seqNum` past the rejecting one for next time.
 3. Otherwise, add `sender` to `p2bResponses[slot]`. If `|p2bResponses[slot]| >= [N/2] + 1` and the slot is not yet CHOSEN, mark it CHOSEN, broadcast a `Decision{slot, command}`, and attempt to advance `slotOut` (execute any prefix of CHOSEN slots starting at `slotOut`).

Decision (ACTIVE leader → all PaxosServers)

- **Contents:** `{slot, command}`.
- **When sent:** By the leader the moment a slot reaches CHOSEN status. Also re-sent by the leader to a follower running behind on `slotOut` (see Heartbeat).
- **At PaxosServer on receipt:**
 1. If the slot is already CHOSEN or CLEARED, ignore.
 2. Mark the slot CHOSEN with the given command.
 3. Attempt to advance `slotOut`, executing each newly-stable command and (if this server has the client's address) sending a `PaxosReply`.

Heartbeat (ACTIVE leader → all PaxosServers)

- **Contents:** {ballot, garbageBoundary}.
- **When sent:** Periodically (every HEARTBEAT_MILLIS) by the active leader, via HeartbeatTimer.
- **At PaxosServer on receipt:**
 1. If ballot < promisedBallot, ignore (stale leader).
 2. If ballot > promisedBallot, update promisedBallot, set leaderState = FOLLOWER. The leader's identity is now promisedBallot.address.
 3. Reset missedHeartbeats = 0 (we just heard from the active leader).
 4. **Garbage collection:** Compute safeBoundary = min(message.garbageBoundary, slotOut - 1). Set local garbageBoundary = max(local, safeBoundary). CLEAR every slot ≤ garbageBoundary. The slotOut - 1 cap encodes the local invariant "never garbage-collect anything not yet executed locally"; under correct global execution it never restricts GC.
 5. Reply with HeartbeatReply{ballot, slotOut}.

HeartbeatReply (PaxosServer → ACTIVE leader)

- **Contents:** {ballot, slotOut}.
- **At leader on receipt:**
 1. If ballot != myBallot OR leaderState != ACTIVE, ignore.
 2. Update executedSlots[sender] = max(existing, slotOut).
 3. Recompute garbageBoundary = min over all servers of executedSlots[server]. Newly garbage-collected slots are CLEARED locally.
 4. **Catch-up:** If slotOut < leader's slotOut, send the lagging follower a batch of Decision messages for slots [sender.slotOut, leader.slotOut) (only those that have not been garbage-collected). Slots that have been CLEARED on the leader cannot be sent — see "Limitations" below.

Timers

HeartbeatTimer (active leader)

- **Pattern:** Tick.
- **Contents:** None.
- **Set:** When this server transitions to leaderState = ACTIVE. Reset unconditionally on every fire.
- **On fire:** If still leaderState == ACTIVE, broadcast Heartbeat{myBallot, garbageBoundary} (skipping self). If no longer active, do not reset (let the timer die).

HeartbeatCheckTimer (every server)

- **Pattern:** Tick (with two-window liveness check, mirroring Lab 2's `PingCheckTimer`).
- **Contents:** None.
- **Set:** On server initialization. Reset unconditionally on every fire.
- **On fire:**
 1. If `leaderState == ACTIVE`, ignore (we are the leader; we don't check on ourselves) and reset.
 2. Increment `missedHeartbeats`.
 3. If `missedHeartbeats < 2`, reset the timer and return.
 4. If `missedHeartbeats >= 2`, the leader is presumed dead. Begin a new election: increment `myBallot.seqNum` past `acceptedBallot.seqNum`, transition to `CANDIDATE`, set `acceptedBallot = myBallot`, reset `missedHeartbeats = 0`, clear `p1bResponses`, broadcast `P1A{myBallot}`, set `LeaderElectionTimer`. Then reset the check timer.

Why two windows? A single missed heartbeat is too aggressive; with reasonable network delay we may simply have lost the heartbeat in flight. Two consecutive misses gives the leader a full timer window to be slow before we displace it. This is identical to the Lab 2 ViewServer's `PingCheckTimer` rule.

LeaderElectionTimer (candidate)

- **Pattern:** Discard.
- **Contents:** `{ballot}` — the ballot under which we are running phase 1.
- **Set:** When transitioning to `CANDIDATE`.
- **On fire:**
 1. If `leaderState != CANDIDATE` or `ballot != myBallot`, discard (we already won, or backed off, or moved on).
 2. Otherwise, re-broadcast `P1A{myBallot}` to any servers we have not yet heard a P1B from, and reset the timer with the same ballot.

AcceptTimer (active leader, per slot)

- **Pattern:** Discard.
- **Contents:** `{slot, ballot}`.
- **Set:** When the leader broadcasts a P2A for the slot.
- **On fire:**
 1. If `leaderState != ACTIVE` or `ballot != myBallot`, discard.
 2. If the local slot status is already `CHOSEN` or `CLEARED`, discard.
 3. Otherwise, re-broadcast `P2A` to servers that have not yet sent a P2B for this `(slot, ballot)`, and reset.

Why per-slot, not per-server? A per-slot timer lets the leader give up retransmitting a slot the

moment it is decided, which keeps the message count tight. The cost is more timers in flight, but `slotOut` advancement bounds how long any timer stays live. The slide deck mentions both options; per-slot is preferred here for its tighter completion behavior.

ClientTimer (PaxosClient)

- **Pattern:** Tick.
- **Contents:** None.
- **Set:** On client initialization. Reset on fire.
- **On fire:** If `pendingCommand` is present and no reply has been received, re-broadcast `PaxosRequest` to all servers.

Correctness / Liveness Analysis

Failure Table

Message	Delayed	Dropped	Reordered	Duplicated
PaxosRequest	Client retries via ClientTimer; AMO dedups any extras	Client retries via ClientTimer	Each request is independent and AMO-deduped; order doesn't matter	AMO sequenceNum dedup at leader and <code>app</code>
P1A	LeaderElectionTimer retransmits	LeaderElectionTimer retransmits	Stale P1As have lower ballots and are rejected	Idempotent: same ballot accepted again is a no-op on receiver
P1B	Candidate waits; if quorum delayed long enough, election effectively timed out and the next election with a higher ballot replaces it	Candidate retransmits P1A to silent servers	Out-of-order P1Bs are matched by ballot; stale ones ignored	Storing in <code>p1bResponses</code> map keyed by sender deduplicates
P2A	AcceptTimer retransmits per slot	AcceptTimer retransmits per slot	Stale P2As have lower ballots and are rejected; CHOSEN slots ack without re-accepting	Acceptor stores by slot; same <code>(slot, ballot, command)</code> is idempotent
P2B	AcceptTimer keeps the slot armed; quorum reached eventually	AcceptTimer retransmits P2A to silent servers	Per-slot quorum tracking is order-independent	Set semantics on <code>p2bResponses[slot]</code> deduplicate by sender
Decision	Catch-up via HeartbeatReply will resend missing decisions	Catch-up via HeartbeatReply	CHOSEN status is monotonic; second decision for same slot ignored	Idempotent at receiver
Heartbeat	HeartbeatCheckTimer's two-window rule absorbs one missed heartbeat	Two consecutive misses trigger election; if leader is actually alive, it loses leadership but a new election restores progress	Old heartbeats have lower ballots and are ignored	Multiple heartbeats just refresh <code>heartbeatSeen</code>
HeartbeatReply	GC progress stalls one cycle but resumes on next heartbeat	Same — GC progress is best-effort	Older replies have lower <code>slotOut</code> and don't decrease the recorded value	Idempotent: max-update on <code>executedSlots[sender]</code>

Per-Slot Safety (the Paxos invariant)

For any slot `s` and any two servers `i, j`: if `i` decides `c_i` for slot `s` and `j` decides `c_j` for slot `s`, then `c_i = c_j`.

This holds because:

- Decisions are reached only when a majority quorum of acceptors has accepted the same `(slot, command)` under some ballot `b`.
- Any future ballot `b' > b` that proposes a value for slot `s` must first run phase 1, collecting P1Bs from a majority. By the quorum-intersection property, at least one acceptor in this majority has already accepted `(slot, c)` under `b`, and reports it. The merge rule keeps the highest-ballot value, so `b'` proposes `c` for this slot, not anything else.
- Therefore every successful decision for slot `s` decides the same `c`.

Liveness

Stable leader: Once a single server's ballot is the highest accepted by a majority, no other server can preempt it without a higher ballot, and no other server will spontaneously start an election while heartbeats are arriving. So in steady state, all client commands complete in two message delays (P2A + P2B quorum), which is the PMMC minimum.

Leader failure: If the active leader crashes or partitions away, every follower's `HeartbeatCheckTimer` fails twice in succession and they all become candidates with new ballots. The candidate with the largest `(seqNum, address)` ballot wins the election (deterministically, given fixed addresses and bounded seqNum jitter). Phase 1 fills holes with no-ops, phase 2 re-proposes any in-flight commands, and progress resumes.

Dueling candidates: If two servers become candidates at the same instant, both will broadcast P1A. The one with the higher ballot wins; the other backs off on receipt of the higher ballot. With deterministic Address tie-breaking, this resolves in $O(1)$ rounds. (In pathological synchrony, randomized timer jitter can be added; we keep timers fixed for simplicity per the slide recommendation.)

Catch-up: A reconnecting follower sends a heartbeat reply with its stale `slotOut`. The leader responds with a batch of Decisions covering the gap (subject to garbage-collection limits — see Limitations).

Node Crash Analysis

Active leader crashes: Heartbeats stop. After two missed `HeartbeatCheckTimer` windows, all surviving followers become candidates. The election winner takes over.

Follower crashes: No effect on safety. If the follower was in the leader's current quorum, the leader may still receive P2Bs from the remaining majority. Garbage collection stalls (because `garbageBoundary` is bounded by the dead follower's `executedSlot`), but operations continue.

Minority crash (up to $\lfloor N/2 \rfloor$ servers): Majority remains; protocol continues. GC stalls until the partition is resolved or — in practice — operators reconfigure (out of scope for this lab).

Majority crash: No quorum, no progress. Surviving minority cannot decide new values but does not violate safety. This is the assumed boundary of the fault model.

Partition / Reconnect Analysis

Minority partition: The minority cannot form a quorum, so cannot elect a leader or decide values. They will repeatedly run phase 1 and fail to assemble P1B majorities. No safety violation.

Majority partition: The majority side elects a leader (possibly a new one) and continues making progress. Decisions made on the majority side cannot conflict with anything the minority side has — the minority hasn't decided anything new.

Reconnect: When the partition heals, the minority side's stale leader (if any) sees a higher ballot in incoming heartbeats and backs off. Stale acceptor state on the minority side is harmless: any value they accepted under a smaller ballot is overwritten on the next P2A from the real leader. Catch-up via HeartbeatReply fills in missing decisions, subject to GC.

Conclusion

Goals Achieved

- **Linearizability.** Single-leader serialization plus per-slot Paxos safety guarantees that every replica executes the same command sequence, and the leader's dedup logic plus AMO ensures each client command appears at most once in that sequence.
- **Majority availability.** Both phases of Paxos require only a majority quorum; the heartbeat / election mechanism ensures a stable leader emerges whenever a majority is connected.
- **Optimal message complexity in the steady state.** With a stable leader, each client command costs one P2A + one P2B quorum + one Decision broadcast — the PMMC minimum.
- **Bounded memory via GC.** Once every server has executed a slot, the leader propagates the boundary and all servers CLEAR the entry.
- **Single-server group support.** With N=1, every quorum is trivially the singleton, so phase 1 is a self-handler call and phase 2 is also a self-handler call. A client command completes in one local step, matching Lab 1 performance.

Limitations

- **GC stalls under partition.** A partitioned minority's `executedSlots` cannot advance, so `garbageBoundary` is held back. Memory grows on the connected majority until the partition is resolved. A more sophisticated scheme (full state transfer to lagging nodes) would lift this restriction at the cost of significant additional protocol.
- **Catch-up cannot replay garbage-collected slots.** If a follower has been disconnected long enough that the leader's `garbageBoundary` has advanced past the follower's `slotOut`, the follower cannot be brought back via Decision messages alone. In this lab the GC rule (only CLEAR slots executed by everyone) prevents this from happening; if it did happen, the follower would be permanently behind. This is the known weakness of the simplified GC scheme.
- **No persistence.** Servers do not survive crashes. A server that crashes loses all its acceptor state and cannot rejoin. This means the fault model is "permanent crash" rather than "fail-stop with recovery."
- **Fixed membership.** The set of servers cannot change at runtime. Reconfiguration would require an

additional Paxos instance for membership changes (out of scope).

- **Dueling-leader.** Under perfectly synchronized timers and pathological message scheduling, two servers could repeatedly preempt each other. In practice, deterministic Address tie-breaking and the two-window heartbeat rule make this vanishingly rare; randomized timer jitter would eliminate it entirely.